ALGO TRADING COMPETITION

Participant Guide

29 Hours — One Asset — One Winner

BUILD	SUBMIT	COMPETE
Day 1, 12 PM	Day 2, 7 AM	Day 2, 2–5 PM

1. What Is This?

You will build an automated trading agent — a Python script — that connects to a live market API and decides every 10 seconds whether to buy, sell, or hold a single asset called Asset Alpha.

You have 29 hours from kickoff to submission deadline. At 2 PM on Day 2 the real market opens and every team's agent runs simultaneously for 3 hours. The team with the highest portfolio value at 5 PM wins.

THE KEY INSIGHT	Asset Alpha is not random. There are hidden mathematical patterns in the price data. Teams that find them and build their agent around them will significantly outperform teams that throw off-the-shelf models at raw prices. Your edge comes from analysis, not model size.

2. Timeline

Time	Phase
Day 1 — 12:00 PM	Kickoff
Day 1 — 12–12:30 PM	Onboarding
Day 1, 12:30 PM – 5:00 PM	Build Phase
Day 2 — 11:00 AM---12:00 PM	Reviewing and Judging

Day 2 — 12 PM—1 PM	Speaker Session
Day 2 — 2:00 PM—2:20 PM	Company Speech
Day 2 — 2:30 PM—5:00 PM	Top Teams Present and Award Ceremony

3. Asset Alpha

You receive one asset: Asset Alpha. This is real market price data that has been anonymised — dates stripped, ticker hidden, prices normalised to an opening value of 100. You cannot identify the underlying stock from the data, and you should not try to.

The training file (asset_alpha_training.csv) contains approximately 116,000 rows of 1-minute OHLCV data: Open, High, Low, Close, Volume. Covering roughly 14 months. This is your primary resource for understanding the asset and building your model.

IMPORTANT HINT	The starter notebook computes specific features in Section 1 before showing you the price chart. That ordering is deliberate. Look carefully at those features before you look at anything else. They were not chosen randomly.

During the live competition, Asset Alpha ticks every 10 seconds. The sequence you trade against is not a replay of the training data — it is a fresh sequence with the same underlying statistical behaviour. What you learn from training data applies to the live run.

4. The Sandbox

From Day 1 at 2 PM until your submission deadline at 7 AM Day 2, a sandbox market is running. This is 17 hours of live ticking price data that you can point your agent at right now.

The sandbox uses the same asset, same tick rate (10 seconds), same API structure, and the same underlying patterns as the live competition. It is not a toy environment — it is a real test environment. Use it to:

- Confirm your agent can connect to the API and execute trades
- Watch your signal detector fire in real time and verify it is working correctly
- Test what happens when your agent tries to buy more than it can afford
- Benchmark your strategy live before the stakes are real

CRITICAL	Run the starter notebook and get a trade executing on the sandbox before 2 PM on Day 1. Teams that skip this step spend hours debugging HTTP requests instead of building strategy. The onboarding window exists specifically for this.

5. The API

5.1 Your API Key

Your team receives a unique API key at kickoff. Every trade request requires it in the request header. Read requests do not require auth.

```
X-API-Key: YOUR_KEY_HERE
```

5.2 Endpoints

Endpoint	Method	What It Returns
/api/price	GET	{ price, open, high, low, close, volume, tick_number, timestamp }. Updates every 10 seconds.
/api/buy	POST	Body: { "quantity": 10 }. Buys at current close price. Returns updated portfolio.
/api/sell	POST	Body: { "quantity": 10 }. Sells at current close price. Returns updated portfolio.
/api/portfolio	GET	{ cash, shares, net_worth, pnl_pct, last_trade_tick }.
/api/history	GET	All ticks broadcast so far this session. Useful for rolling feature calculation.
/api/leaderboard	GET	Live rankings: [{ team_name, pnl_pct, rank }]. Public, no key needed.

5.3 Rate Limits

- Maximum 1 trade per 10-second tick. Attempting more returns HTTP 429 — your agent must handle this.
- Read endpoints are unlimited but throttled at 10 requests/second. Polling once per tick (every 10 seconds) will never hit this.
- Your agent should sleep 10 seconds between polls. Tighter polling gains nothing and risks triggering the throttle.

6. Portfolio Rules

Rule	Detail
Starting balance	100,000 virtual USD cash. 0 shares.

Transaction fee	0.1% of trade value on every buy and sell. Applies to both directions.
Cash decay	0.02% per minute on cash during the live run only (2–5 PM). Over 3 hours this is ~3.6% on fully undeployed cash. A gentle pressure, not a punishment.
Maximum position	60% of your current net worth in shares at any time. Server enforces this.
Short selling	Not permitted. You can only sell shares you already hold.
Fractional shares	Not permitted. Quantity must be a positive whole number.

Why cash decays but stock does not

Cash decay represents the opportunity cost of uninvested capital — idle money loses purchasing power. There is no real-world analogue for a holding fee on stock you own outright, so none has been invented. What makes passive holding a losing strategy is the price series itself: the market ends the session below where it opened. Any agent that buys at the start and holds to close will be in the red — not because of a rule, but because the market went down.

7. Win Condition

```
Net Worth = Cash Balance + (Shares Held × Final Tick Price)
```

At 5:00 PM all open positions are valued at the last broadcast price. The team with the highest net worth wins.

Tiebreaker: if two teams finish within 2% of each other, the tiebreaker is Sharpe ratio — your mean return per trade divided by the standard deviation of your per-trade returns. Consistent, controlled trading beats one lucky all-in bet.

8. The Starter Notebook

You receive a Jupyter notebook (starter.ipynb) at kickoff. Run it before writing any original code. It contains:

- Section 1 — Data loading and feature visualisation. Loads the training CSV, computes and plots the features that matter. Run this first and look carefully at what is being computed.
- Section 2 — Feature engineering library. Pre-built functions you can use and extend.
- Section 3 — A fully working SMA crossover agent. Generates signals, sizes positions, applies a stop-loss, calls the API. Makes real trades on the sandbox. Must run without modification.

- Section 4 — Backtesting scaffold. Pass it any signal function and get back P&L, trade count, and Sharpe ratio on the training data. Use this to evaluate every change before deploying live.
- Section 5 — Blank strategy cell. Your starting point.

NOTE	The starter strategy has a deliberate weakness. It will not win the competition as submitted. Understanding why it underperforms and fixing it is the task. The backtester in Section 4 will show you exactly where it bleeds.

9. Submission Requirements

Deadline: 7:00 AM, Day 2. No exceptions.

Submit via the Google Form link shared at kickoff. Your package must include:

- agent.py — a single runnable Python file. If you need extra libraries, include a requirements.txt.
- README.txt — plain English description of your strategy. What pattern did you find? How did you size positions? How did you manage risk? This is read during the showcase and reviewed before the live run.

Technical requirements for agent.py:

- Reads API_URL and TEAM_API_KEY from environment variables (do not hardcode either)
- Runs with: python agent.py
- Handles HTTP errors gracefully — a single failed request must not crash the agent
- Terminates cleanly on KeyboardInterrupt

ENV VARS	Your agent will be launched as: API_URL=http://live-server TEAM_API_KEY=your-key python agent.py If you hardcode the URL or key it will break during the live run when pointed at the official server.

10. Rules

Allowed

- Any Python library installable via pip
- Any trading strategy, ML model, or statistical method you can implement and explain
- All competition API endpoints
- The full training CSV for any purpose: feature engineering, model training, backtesting

Prohibited — Immediate Disqualification

- Fetching live or real-time market data from any external source during the live run
- Hardcoding trades based on specific tick numbers — your agent must make decisions from data
- Sharing your API key with another team or coordinating positions during the live run
- Reading files outside your agent's working directory

NOTE	Submitted code is reviewed between 7 AM and 12 PM on Day 2. Keep your code readable. If your README describes a momentum strategy but your code contains external HTTP calls, that is a disqualification.

12. Tips

- **Look at Section 1 of the notebook before you look at the raw price chart. The features computed there were chosen deliberately. They are the breadcrumb trail.**
- The API returns Open, High, Low, Close, Volume on every tick. Teams that only use Close are ignoring 80% of the available data.
- Backtest every change before you deploy it live. The scaffold in Section 4 takes 10 minutes to use and will save hours of wasted sandbox runs.
- Size your positions. Going all-in might work. A strategy with deliberate position sizing and a stop-loss is more likely to survive the full 3 hours and will score better on the Sharpe ratio tiebreaker.
- Test your error handling. Your agent runs unsupervised for 3 hours. What happens if the server returns a 429? What if you try to sell more shares than you own? Handle these or your agent silently stops trading mid-competition.
- The leaderboard updates every 30 seconds. You cannot change your submitted code during the live run. Watching the leaderboard is interesting but acting on it is pointless. Trust your backtest.
- Sleep before the live run. An agent written at 6 AM on no sleep will have bugs that an agent written at midnight with 5 hours of sleep will not.

The market has a pattern.
Find it.

Good luck.